Technical report: booklet-check

- **Track:** Track 2A, OST challenge: Multilingual Natural Language Inference over Swiss Official Voting Booklets
- **Event:** Online
- **Team:** BookletCheck (Hua-I Yu, solo)
- **Demo:** `make run` in `track_2a/` (Docker, two official example cases); full numbers in `eval/RESULTS.md`

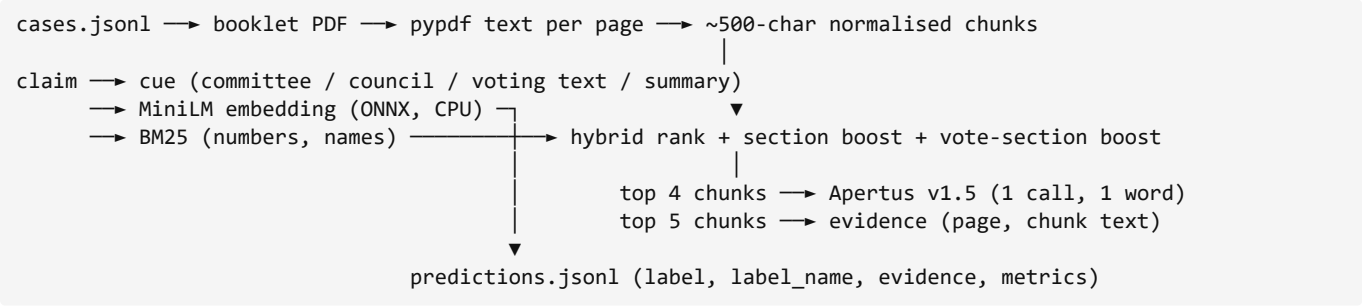
1. Summary

A claim about a Swiss federal vote, in German, French or Italian, has to be labelled entailment, neutral or contradiction against the official voting booklet: either the whole PDF (task A) or one given passage (task B). Task A also asks for evidence with page numbers.

Our system keeps the language model on the one job only it can do. Plain code extracts the text of each PDF page with pypdf, normalises it (NFKC, repeated spaces and blank lines removed), cuts it into chunks of about 500 characters, and ranks them with a small multilingual embedding model, BM25 and a structural prior (claims name their source, such as "the committee says" or "according to the voting text", and the booklet sections can be recognised from their page headers). Apertus v1.5 then reads only the top four chunks and answers with a single word. The evidence we return is the five best chunks themselves, with the page they were cut from, so the model cannot invent a quote and the page is always the one the text came from.

On the 5 held-out vote dates (389 task A and 389 task B cases), apertus-v1.5-8b with booklet-check reaches macro-F1 0.895 on task A (evidence score 0.804) and 0.949 on task B, with 812 input tokens per task A case. On the same sample of 60 task A cases, the whole-booklet baseline scores 0.737 (evidence 0.275) with 38,574 input tokens per case, against 0.917 (evidence 0.900) with 821 for ours. For 18 of these cases the booklet was cut at 180,000 characters to fit the context, but that is not the main reason for the gap: on the 42 uncut cases the baseline gets 31 of 42 labels right and ours 40, on the 18 cut ones 14 against 15. On task B, where the passage is given, the baseline is slightly ahead (0.967 against 0.950, 58 against 57 correct). With 60 cases per task the uncertainty is large: the 95% bootstrap interval of our task A advantage is +0.07 to +0.30 macro-F1.

2. Architecture



- `src/booklet_check/pdf_index.py`: page extraction (pypdf), NFKC normalisation and whitespace clean-up, then chunking along line breaks (blank lines are dropped). Every chunk comes from one page, so evidence page numbers are exact.
- `src/booklet_check/retrieve.py`: hybrid ranking. The score of a chunk is its cosine similarity (multilingual MiniLM), plus 0.35 times its normalised BM25 score, plus 0.08 if its page belongs to the vote named in the case, plus 0.3 if its page is in the section the claim names. Section tags come from page headers ("Argumente Komitee", "Arguments du Conseil fédéral", "Testo in votazione", ...), in all three languages and in both booklet layouts used since 2020.
- `src/booklet_check/pipeline.py`: prompt, label parsing, evidence. Task B uses the same prompt on the given passage; passages longer than 2,600 characters are cut down to their best chunks first.
- `src/booklet_check/llm.py`: thin OpenAI-compatible adapter. It reads `BASE_URL` and `API_KEY` (the names the evaluation injects; the template's `LLM_*` names also work), retries 429 and 5xx with backoff, and records real token counts. With no key it runs a clearly labelled DRY MODE that makes no calls, so `make run` works on a clean checkout.
- Parallelism: retrieval runs under a lock (CPU), model calls run in 4 worker threads.

Cross-lingual claims (for example a German claim about the French booklet) are the reason for the embedding model: BM25 cannot match across languages, while numbers, dates and names still help, which is why both are combined.

3. Use of Apertus

- **Model:** `swiss-ai/apertus-v1.5-8b` (instruct) for every reported run. We tried 70B, but Public AI answered 429 (rate limited), so there is no 70B result. The endpoint's model list is read at start-up when `LLM_NAME` is not set.
- **How it is used:** inference only, one call per case, temperature 0, `max_tokens` 8.
- **Where it runs:** hosted OpenAI-compatible endpoint (Public AI during development; the organisers' proxy during evaluation).
- **Prompt:** a one-line system message ("You verify claims against the official Swiss federal voting booklet. Answer with exactly one word"), then the vote title, the passages with their page numbers, the claim with its language, and short definitions of the three labels (contradiction explicitly includes a different number, date, amount, actor, consequence, position or recommendation). See `pipeline.py`.

Other models. The only other model is `sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2` (Apache-2.0, 8-bit ONNX, 118 MB) for retrieval. It never sees a label and never produces text; it only orders the chunks. It is downloaded when the Docker image is built (pinned revision, sha256 checked) and read from disk at run time; a CI step runs the image with networking disabled. We also tried `multilingual-e5-small` and `multilingual-e5-base` in the same slot; on the development split neither found clearly more gold passages, so the smallest and fastest model stayed. No other model was used as a judge.

4. Data

- `OSTswiss/MNLIoverSwissVotingBooklets` v1.1 (MIT), 1,488 rows over 20 vote dates, three labels in equal parts, nine language pairs. Cases and gold files are produced with the official `prepare_cases.py`; booklet PDFs come from the Federal Chancellery.
- **Held-out split** (`data/split.json`, `scripts/make_split.py`): by vote date, so that held-out booklets are never seen during development, as the private test set may contain unseen booklets. The split is deterministic: a hash of each date picks 5 of the 20 dates (389 rows, 778 cases) for the held-out set, and the rest (1,099 rows) is for development. The rule and the dataset hash are stored in the split file.
- **Timeline** (2026-10-14, UTC+8, from git and `eval/spend.jsonl`). The first development model call was at 22:03:56, before the split file was committed in `0fd4169` at 22:49:32. No development run used a held-out case (0 held-out rows across all development runs in `eval/runs/`, which `make_results.py --check` recomputes; the smoke test before them also used development dates only). The system was frozen in commit `7b6d3a1` at 23:13:51, and the first held-out call came after that, at 23:14:27.
- Two observations from the development data shaped the design: neutral claims are claims written for another vote, and the gold passage of a claim is almost always the booklet section that the claim's opening words name (summary, committee, Federal Council, voting text).
- Nothing personal is stored. The repository holds the two official example cases and their booklet; other PDFs are downloaded by `prepare_cases.py --download-booklets`.

5. Evaluation

All scores are computed by the official `evaluate.py` (starter commit `5e69577`) on our prediction files. Every number in this report is read from the files in `eval/` by `scripts/make_results.py`; `eval/RESULTS.md` has all runs, including development ablations.

The baseline is the same Apertus model with the whole booklet in the prompt (no retrieval), asked for the label, the page and a verbatim quote (`scripts/baseline_full_context.py`). Because of the credit available, the baseline was run on a label-balanced sample of the held-out set; our system was run on the same sample and on the full held-out set.

run	system	model	cases A / B	A macro-F1	A evidence	B macro-F1	input tokens per case A / B	seconds per case A / B	USD
`holdout-ours-8b`	ours	apertus-v1.5-8b	389 / 389	0.895	0.804	0.949	812 / 750	4.5 / 4.6	0.0613
`holdout60-baseline-8b`	baseline	apertus-v1.5-8b	60 / 60	0.737	0.275	0.967	38,574 / 2,096	14.0 / 5.8	0.2451
`holdout60-ours-8b`	ours	apertus-v1.5-8b	60 / 60	0.917	0.900	0.950	821 / 830	10.1 / 6.3	0.0100

Against the baseline. On the 60 task A cases ours gets 55 labels right and the baseline 45. Cutting long booklets is not what separates them: on the 42 uncut cases it is 40 against 31, on the 18 cut ones 15 against 14. Most baseline errors are missed contradictions (recall 0.45, ours 0.85). On task B, where both systems see the same passage, the baseline is ahead: 0.967 against 0.950 (58 against 57 correct). This suggests that our task A gain comes from finding the right passage, not from reading it better. With 60 cases per task these numbers are rough. The 95% bootstrap intervals (10,000 paired resamples of cases, `eval/analysis.json`) are 0.84 to 0.98 for our task A macro-F1 and 0.62 to 0.84 for the baseline; the difference is +0.07 to +0.30 on task A and −0.06 to +0.00 on task B. We claim fewer input tokens and a better task A score on this sample, not a faster system: the 10.1 s against 14.0 s per task A case were measured on a shared public endpoint with queueing and do not compare speed.

Held-out macro-F1 of `holdout-ours-8b` by language pair (booklet or passage language, then claim language):

pair	de->de	de->fr	de->it	fr->de	fr->fr	fr->it	it->de	it->fr	it->it
task A	0.97 (n=34)	0.72 (n=36)	0.72 (n=40)	1.00 (n=55)	0.91 (n=37)	0.91 (n=50)	0.80 (n=47)	0.98 (n=37)	0.98 (n=53)
task B	0.87 (n=34)	0.88 (n=36)	0.88 (n=40)	1.00 (n=55)	1.00 (n=37)	0.98 (n=50)	0.94 (n=47)	1.00 (n=37)	0.93 (n=53)

Each cell holds only 34 to 55 cases from 5 vote dates, so one case moves a cell's accuracy by 0.02 to 0.03 and the table should be read as a rough picture. Two patterns stand out. In task A the weakest pairs are German booklets with French or Italian claims (26 of 36 and 29 of 40 labels right). They also have the lowest evidence hit rate (17 of 27 and 18 of 34 cases with the gold passage among our five chunks, against 0.87 on the other pairs), while the same pairs reach 0.88 and 0.88 in task B, where the passage is given. So most of the loss there is cross-lingual retrieval, not reading. In task B the weakest cell is de->de (30 of 34), and German passages account for 12 of the 20 task B errors while making up 110 of the 389 cases; we have not found a single cause for that.

Development choices, all on development dates only (full list in `eval/RESULTS.md`). On a 150-case task A sample, showing Apertus 8 chunks gave macro-F1 0.966 (145 right), 4 chunks 0.953 (143) and 6 chunks 0.932 (140; 6 of its cases were lost to a local DNS outage and count as wrong). Going from 8 to 4 cost 0.013 macro-F1, but at this sample size that is noise: the 95% bootstrap interval of 8 minus 4 is −0.01 to +0.04. We kept 4 because it uses 45% fewer input tokens than 8, not because it scored better. The section prior raised the share of cases whose five evidence chunks contain the gold passage, without any model call.

Running it. The Docker image is 425 MB uncompressed, and a cold build took 21 s on a GitHub-hosted ubuntu-24.04 runner (CI run 38594214359, `eval/docker.json`); `make run` on the two example cases in DRY MODE then took 5 s. We did not time the 778 held-out cases inside the container, because that needs model calls. The same code needed 595 s (about 10 minutes, 4 workers) for them on the development laptop against Public AI (`wall_s` in `eval/results.json`). Our estimate for the container is the same order plus a few seconds of start-up, but that is an estimate, not a measurement.

6. Limitations

- Evidence is retrieved, not chosen by the model. When the information is repeated in two sections (the summary and the detailed section often say the same thing), our chunk can be right but outside the gold passage, which costs evidence score but not label accuracy.
- Cross-lingual retrieval is the weak spot (German booklet, French or Italian claim, see section 5).
- The section prior relies on header wording seen in 2020 to 2026 booklets. A new layout would silently turn the boost off (no wrong boost, just none).
- Romansh is not supported and was not tested. The public data has no Romansh cases, and neither the BM25 stop words nor the section-header rules cover it.
- Neutral claims in this dataset come from other votes. If the private set has neutral claims about the same vote that the booklet simply does not settle, the model must recognise that from the passages; we did not hard-code a similarity cut-off for this reason.
- The embedding model is the 8-bit AVX2 ONNX export. On a CPU without AVX2 it may not load; the system then prints a warning on stderr and continues with BM25 only, which hurts cross-lingual claims. CI fails on that warning, but a run elsewhere would not stop.
- pypdf text of some older booklets splits words at line ends; the scorer tolerates this, but our BM25 sees two tokens.
- Development numbers come from 8B on a public endpoint with variable latency, so the time per case says little about the official proxy.

7. Reproducibility

```
cd track_2a
make run                                    # Docker, linux/amd64, CPU only; DRY MODE without a key
BASE_URL=... API_KEY=... make run           # real run on data/examples
make run INPUT_DIR=/path/to/data INPUT=cases.jsonl   # any official case file (+ booklets/)
```

`prepare_cases.py` (cases, gold labels, booklet download) and `evaluate.py` (scorer) are not in this repository. They come from the official starter, https://gitlab.com/ifsoftware/hackapertus-starter, at commit `5e69577`. Our `requirements-dev.txt` also covers the scorer's dependencies (rapidfuzz, scikit-learn); `prepare_cases.py` needs only the standard library. Evaluation on the held-out split (Python 3.12):

```
git clone https://gitlab.com/ifsoftware/hackapertus-starter starter && git -C starter checkout 5e69577
cd track_2a && pip install -r requirements-dev.txt
python docker/fetch_model.py models/minilm                      # embedding model, sha256-checked
python ../starter/prepare_cases.py --download-booklets --output-dir full
python scripts/split_cases.py --cases full/cases.jsonl --expected full/expected-labels.jsonl \
       --part holdout --out-dir work/holdout
APERTUS_BASE_URL=... APERTUS_API_KEY=... EMBED_DIR=models/minilm \
python scripts/run_eval.py --system ours --split-dir work/holdout --booklet-root full \
       --tag holdout-ours --official-eval ../starter/evaluate.py
python scripts/make_analysis.py --work work --official-eval ../starter/evaluate.py
python scripts/make_results.py
```

Samples: `work/holdout60` is cut with `--part holdout --ids eval/runs/holdout60-ours-8b/predictions.jsonl`, `work/devA150b` with `--part dev --task A --sample 150 --seed 3`. Temperature is 0 and there is no sampling elsewhere, so reruns differ only by endpoint non-determinism. CI (`.github/workflows/ci.yml`) runs the unit tests and `make_results.py --check`, builds the image, runs `make run` and checks the output contract, and reruns the image with networking disabled.

8. Next steps

- Let Apertus pick which of the five evidence chunks decides the case (one extra token) and put it first.
- Improve cross-lingual retrieval for German booklets, for example with a cached claim translation when retrieval is unsure.
- Learn the section prior from the development data instead of header rules.

License

Creative Commons Attribution 4.0 (CC-BY-4.0). Code is Apache-2.0, data we add is CDLA-Permissive-2.0. All Hack Apertus projects are open source.

References

- OST dataset: https://huggingface.co/datasets/OSTswiss/MNLIoverSwissVotingBooklets
- Official starter and evaluator: https://gitlab.com/ifsoftware/hackapertus-starter
- Apertus: https://huggingface.co/swiss-ai
- Reimers and Gurevych (2020), Making Monolingual Sentence Embeddings Multilingual using Knowledge Distillation (paraphrase-multilingual-MiniLM).
- Robertson and Zaragoza (2009), The Probabilistic Relevance Framework: BM25 and Beyond.